

CSC 487 HW 2 Report – Facial Keypoints

Code Explanation

In order to predict the facial keypoints from images loaded and annotated from an external dataset, I used a CNN in addition to some data preprocessing to prep it for the model. Firstly, I used a 90/10 train/test split for validation during training. I then scaled the image and keypoint values. I noticed that there were no nan values in the images, so I simply scaled the image values down from [0-255] to [0-1] by dividing all the images by 255, so that the learning would go smoother (due to smaller values with respect to the CNN parameter values). Since the keypoints had many missing (nan) values, I used the MinMaxScaler from sklearn to scale down the keypoint values from about [0-96] (the pixel location values on the image) down to [0-1], for the same reason as the images. This allowed me not to have to worry about the nans when scaling these values down.

Due to the large amount of images, I loaded in the train and test sets in TensorDatasets and then DataLoaders using pytorch in order to use standard batches of 32 during training. For my CNN architecture, I decided to adhere to the advice of “don’t be a hero” and used a VGG-style architecture, with three sets of 2 conv and 1 Max Pooling layers (with ReLUs in between), before flattening out the network into a single-layered MLP, similar to in our previous CNN lab.

Due to the nans within the keypoint labels, a custom loss function was written to handle this, and I decided to stick with mean absolute error (MAE) for simplicity’s sake. Adam was the optimizer of choice then used for training, and 50 epochs was chosen for training, as plotting the train and test losses over the epochs showed that the losses tended to plateau before 50 epochs. The learning rate and weight decay of $1e-3$ and $1e-4$, respectively, were chosen due to typical success in the past for various models.

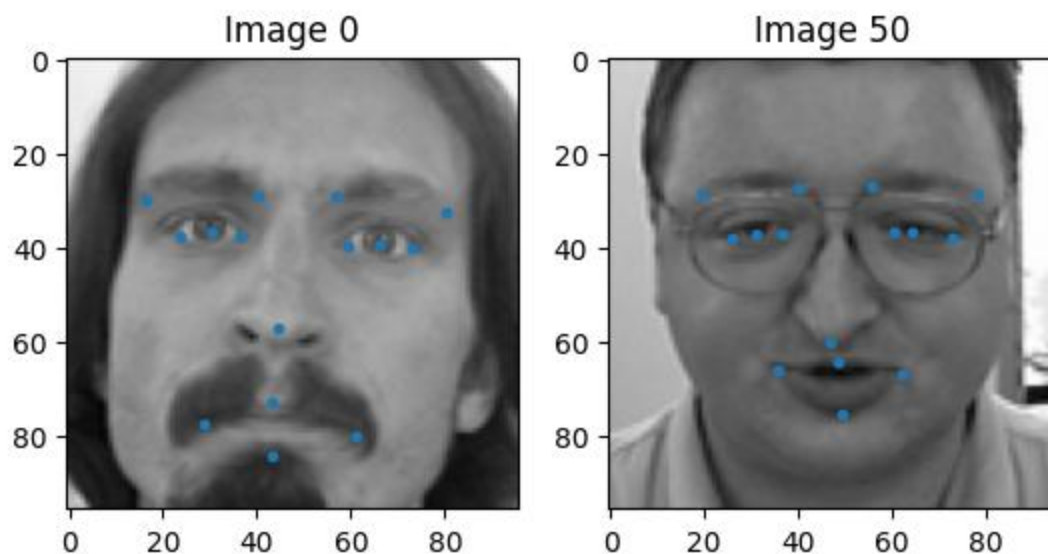
I mostly used the sklearn and pytorch documentation/APIs for guidance when completing this assignment, especially the examples for how the different classes and functions I needed were used, as well as which classes and functions were even available. For example, using the sklearn docs to figure out how to apply the MinMaxScaler. I also used Perplexity (AI) for double checking various bits of my code, such as code snippets for plotting my images and keypoints to analyze my model’s results, along with help diagnosing issues arising from using the GPU through torch and switching back to the CPU for results analysis.

Discussion

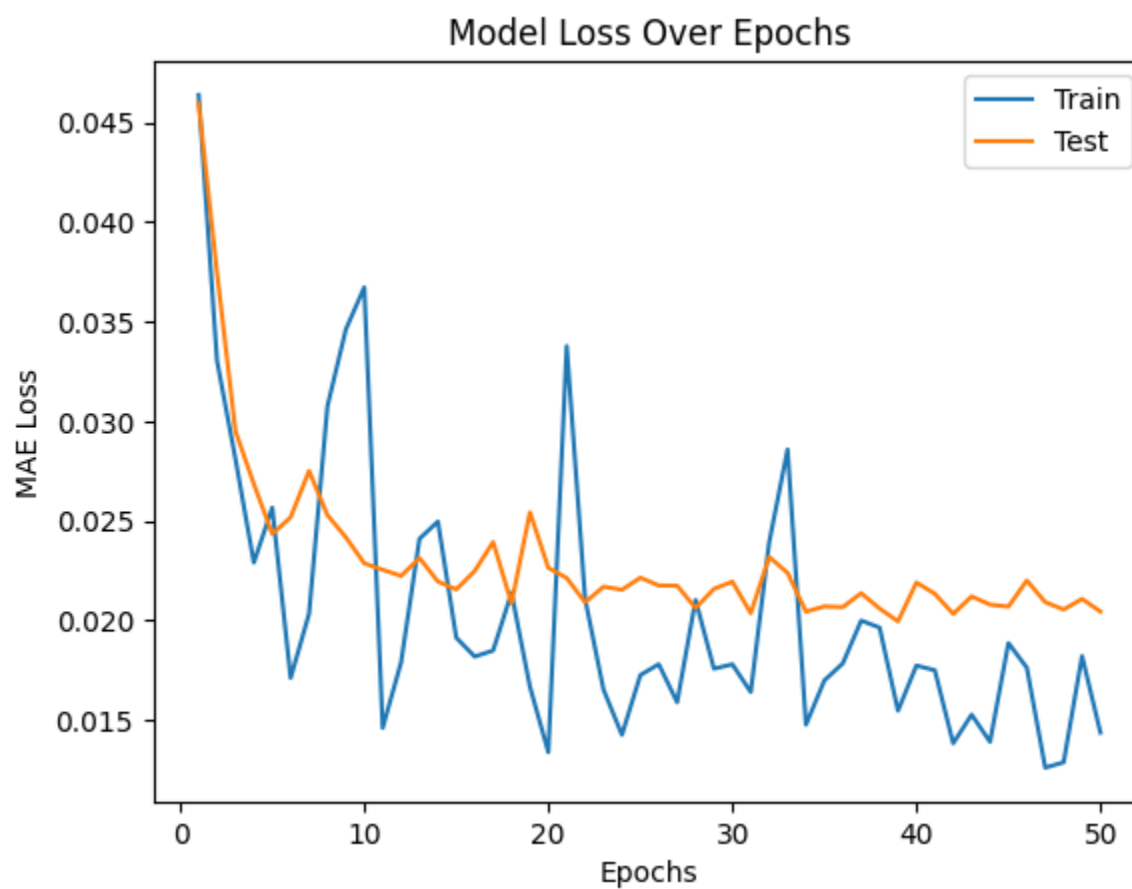
The imported data had two components we used: *images* and *keypoints*, both of which were np arrays. These arrays had the following properties:

- *Images*
 - Shape: (7049, 1, 96, 96)
 - This refers to 7049 images, each with 1 channel (grayscale) and 96 by 96 pixels
 - Dtype: int64
 - Min: 0
 - Max: 255
 - Interestingly, the int64 dtype is used for the *images array* are of dtype int64, even though the pixel values are bounded between 0-255, i.e. the standard uint8 range of values for RGB/grayscale images.
- *Keypoints*
 - Shape: (7049, 30)
 - This refers to 7049 sets of 30 values, representing the x & y pixel locations for the 15 keypoints—i.e., the ground truth location for each of the keypoints for each of the images in *images*
 - Dtype: float32
 - Min: 0.687
 - Max: 95.936
 - The min and max values indicate that the keypoint values represent the values of the pixel's x and y coordinates on the image

Examples of these images with the keypoints plotted are shown below:



Training the CNN yielded the following loss curves for the train and test set.



While the train set especially seems to have lots of noise in its loss, overall both the train and test sets see their MAE losses decrease steeply and then gradually with the epochs,

before essentially flattening out after 20 epochs or so. This “smooth” loss shape, especially on the test set, seems to indicate that the learning rate chosen was suitable for this application. The train MAE seems to plateau at about 0.015, which corresponds to an average keypoint error of about 1.4 pixels (0.015×96) since the MAE is scaled down to essentially fit into $[0,1]$ from its original range of about $[0,96]$. Similarly, the test MAE seems to plateau at about 0.020, or roughly 1.9 pixels of average keypoint error. This small pixel error seems to indicate a relatively small amount of overfitting having occurred, and a 1.5-2 pixel error seems to indicate relatively low bias as well.

Upon training the model, 4 random images were selected from the train and test sets, and then the predicted and ground truth keypoints were plotted on said images. The 4 train set images and their keypoints are shown below.

Train Set Example Images With Predicted Keypoints

Image 6319

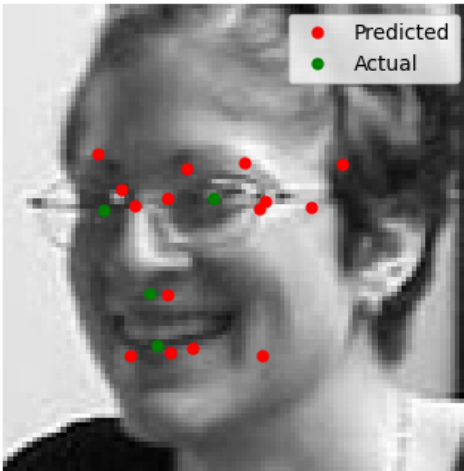


Image 4318

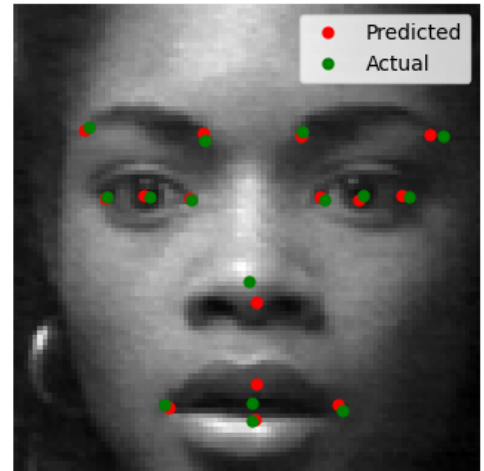


Image 6095

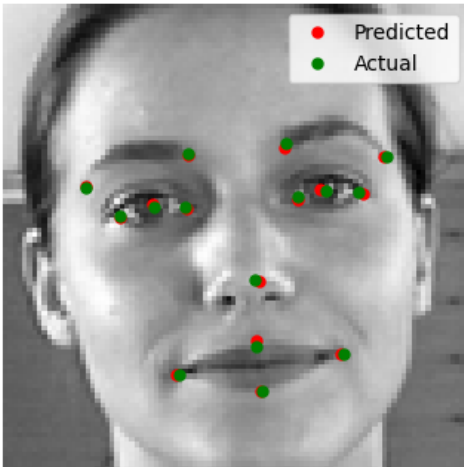
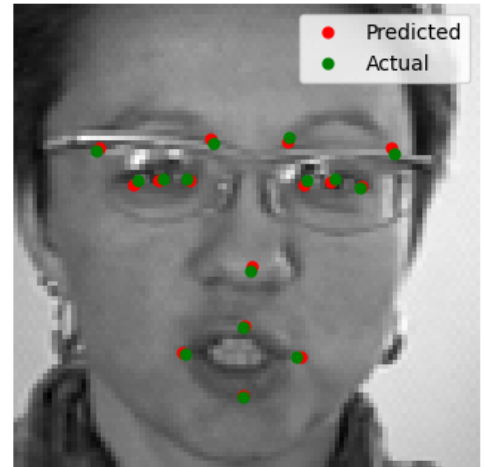


Image 2577



The four random test set images and their keypoints are also shown below.

Test Set Example Images With Predicted Keypoints

Image 38

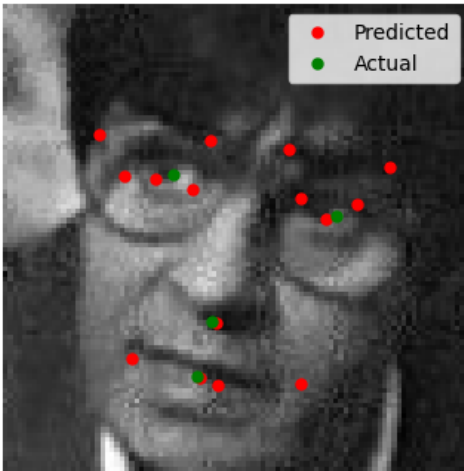


Image 409

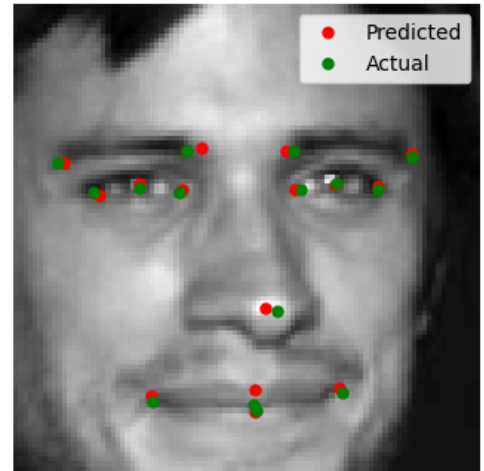


Image 642

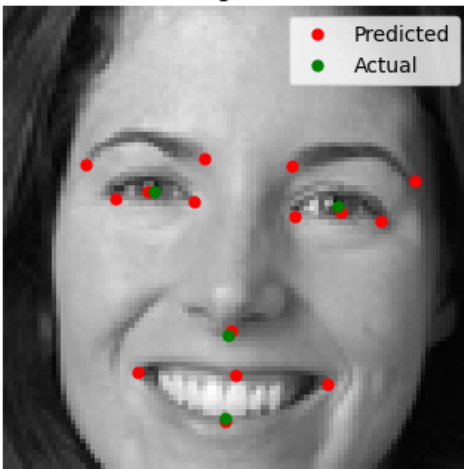
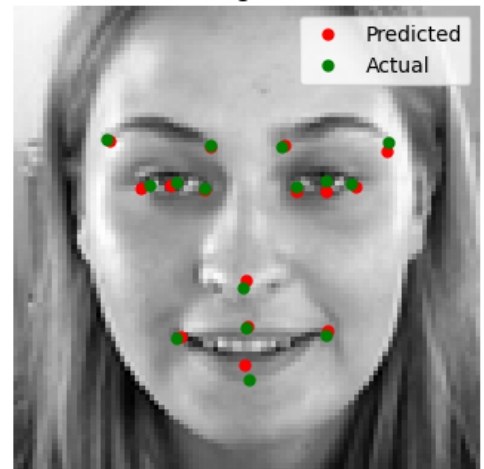


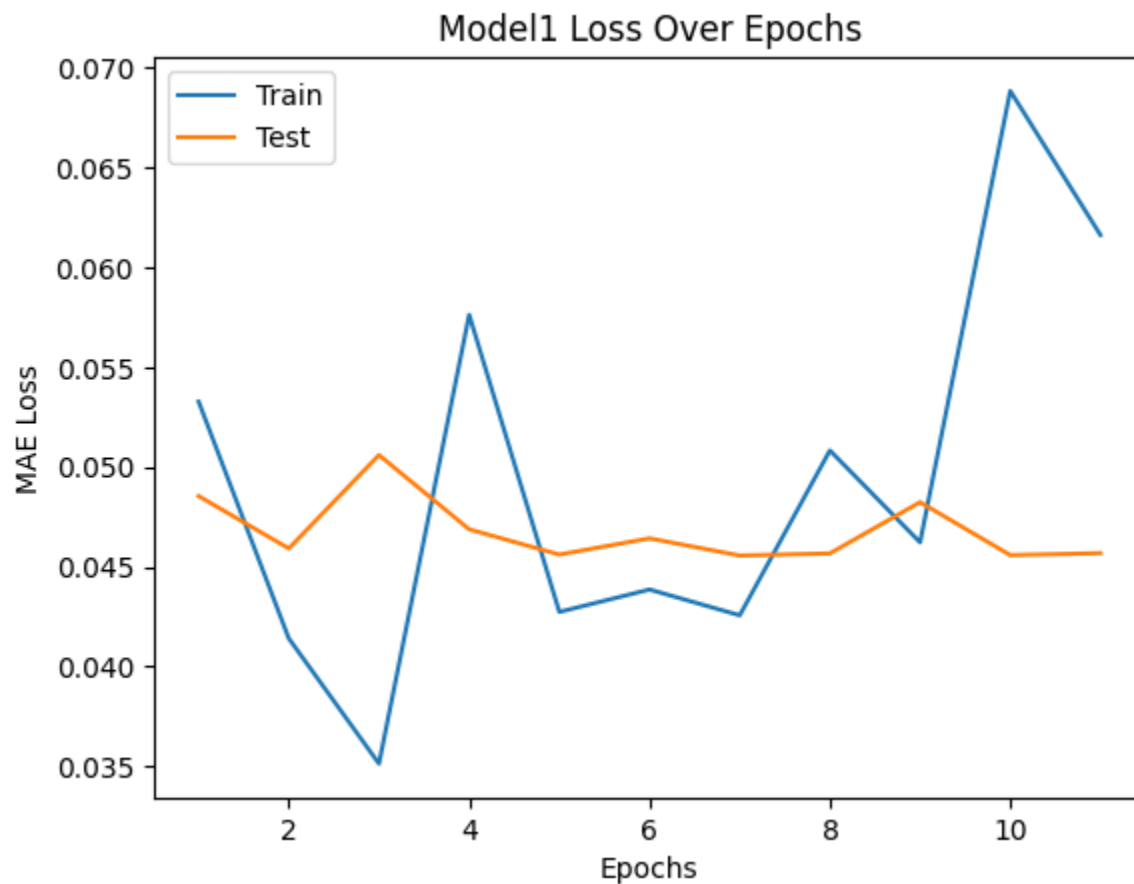
Image 99



Further supporting the notion that the model is not overfitting, the predicted vs actual keypoints plotted on these random images looks very similar between the train and test sets. Additionally, the model seems to struggle relatively more with predicting keypoints for images that had many missing ground truth keypoint labels—this was the case for both train and test set images, however.

In order to try and improve the model, two modifications were tried to the CNN architecture. The first change was regarding the multi-layer perceptron (MLP) portion of the CNN, which in the original model just went straight from the flatten layer to the predictions for each keypoint. Thus, I added one hidden layer with double the neurons as the flattened layer to stay true to the VGG-style architecture. This, however, not only severely slowed

down model training (despite still using the same GPU for computation via Colab) but also seemed to severely hurt model performance too, causing this new model ("Model1") to not be decreasing in loss even after several epochs, as seen in the plot below.



With a train loss of over 0.050 which did not seem to be decreasing with more epochs, this meant that predicted keypoints were on average about 5 pixels away from ground truth, which is much worse than before. When plotted on their corresponding images, test set keypoints were also noticeably much farther on this model than the originals', as seen below.

Model 1 (More MLP) Test Set Example Images With Predicted Keypoints

Image 281

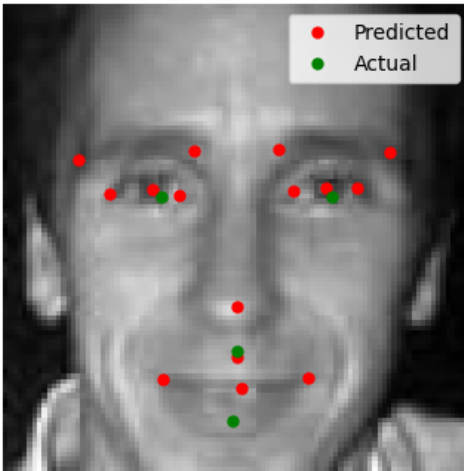


Image 272

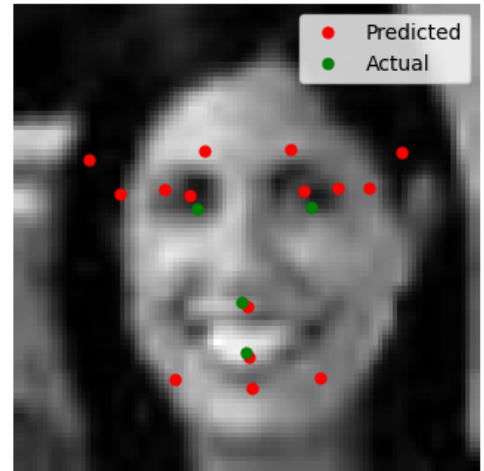


Image 344

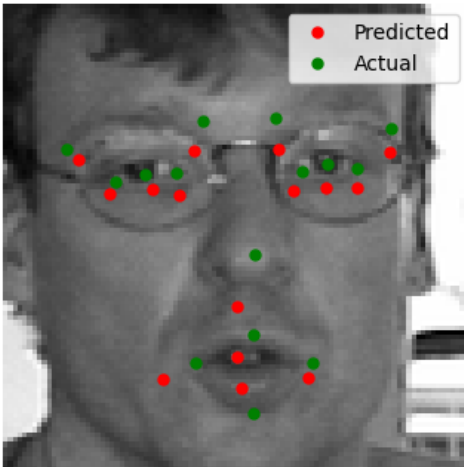
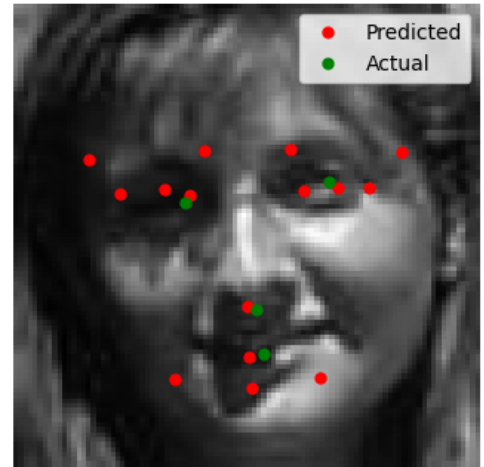
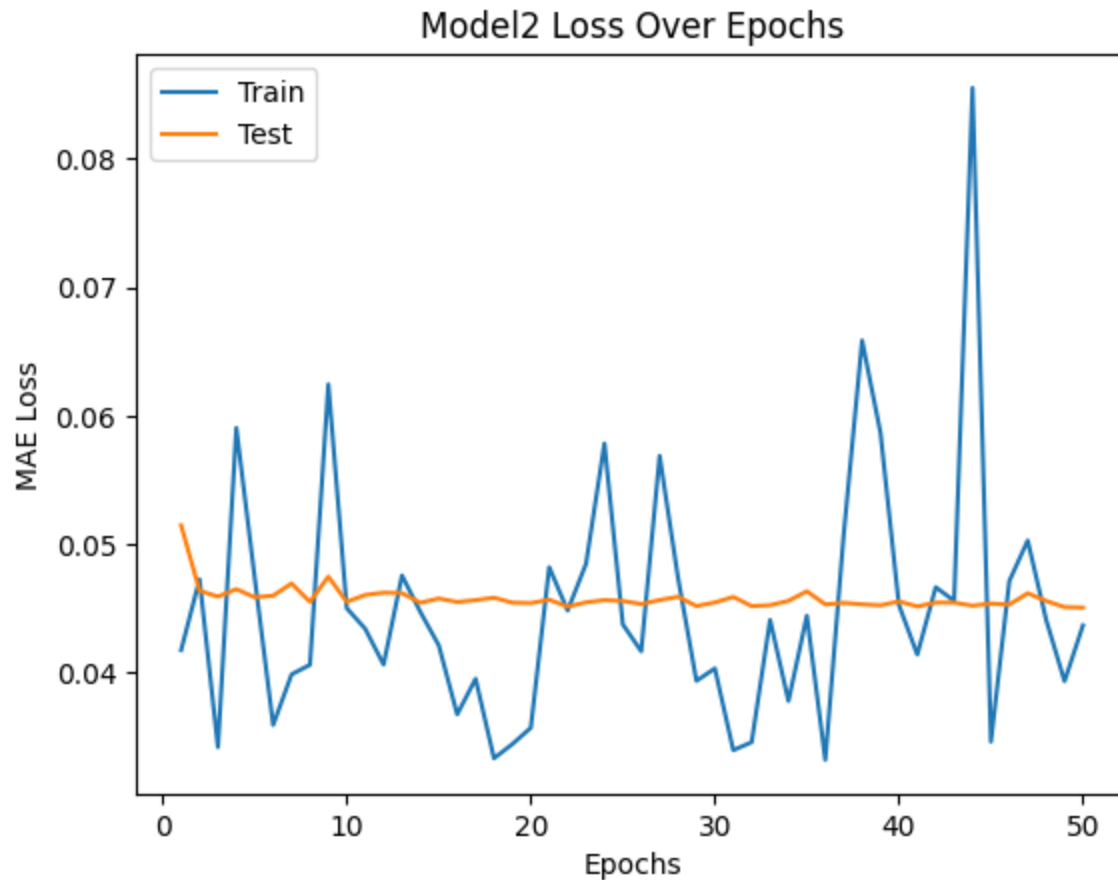


Image 408



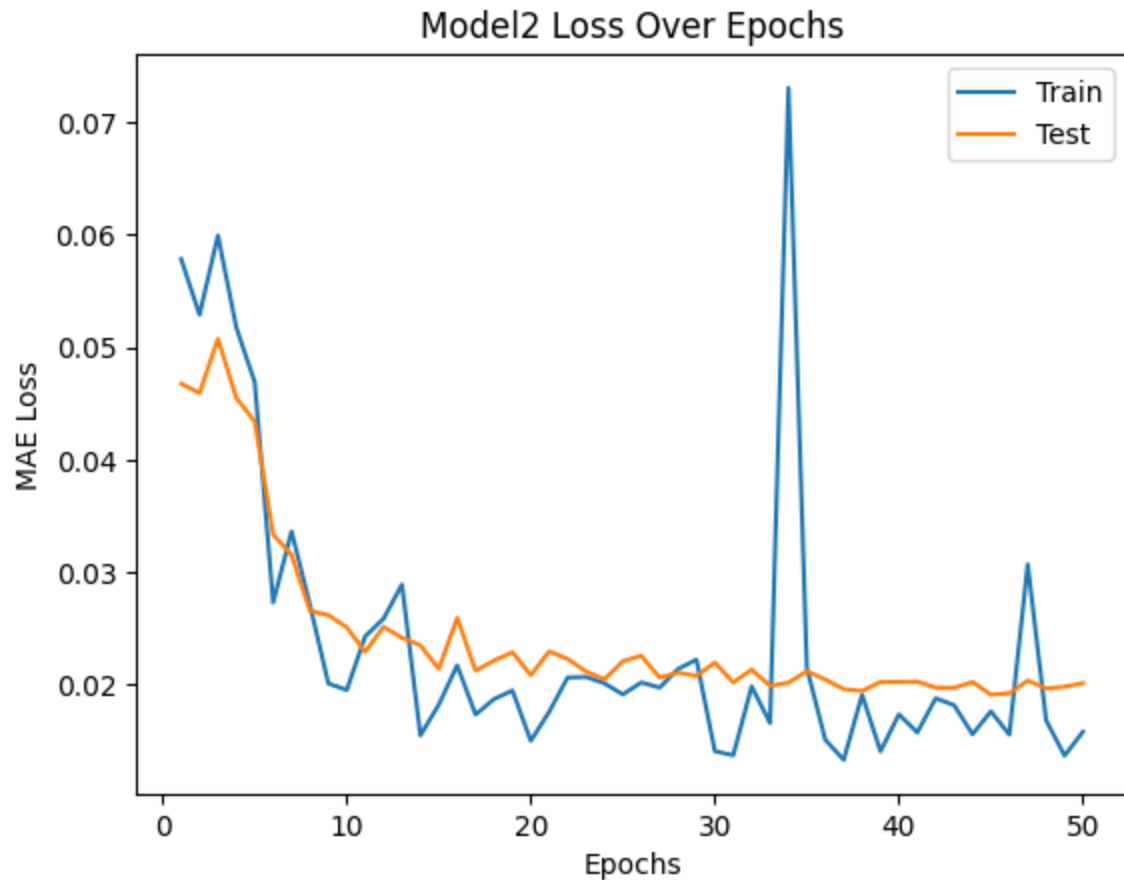
As a result of these shortcomings, this costly training was cut short and the second potential improvement to the model was tried—namely adding more VGG-style conv-conv-maxpool sections to the CNN, and returning the MLP portion back to my original simple linear classifier.

Adding two more conv-conv-maxpool sections was first tried, though this also seemed to worsen performance. Resulting in final train and test losses (average keypoint error) of about 4-5 pixels for both.



Upon further review, it seemed that not much was learned by the network, perhaps due to a vanishing gradient problem from the extra layers but not adding in aspects of a residual neural network (NN). Furthermore, the extra two maxpool layers added meant that by the end of the CNN portion the images had been down sampled to merely 3x3 pixels. Since the keypoints are mapped to small, specific regions on the face, this seemed too small to yield the spatially precise results I hoped to achieve, so I removed one of these sections for the next model tried—i.e., four conv-conv-maxpool sections which resulted in a 6x6 output image.

Trying with only the one additional conv-conv-maxpool section resulted in the following train/test loss over the training duration. This was a similar amount of bias and variance to the original model, with both train/test MAE plateauing again around 0.20, translating to about 2 pixels off from ground truth for the keypoint predictions.



Lastly, I tried actually removing a conv-conv-maxpool section from the CNN to see if making the linear regression “decisions” based on a less down-sampled image would yield more accurate keypoints. This last model’s train/test loss during training can be seen below. In short, removing this section hurt test set performance, which was now plateaued around 0.025 MAE (~2.5 pixels average error). Thus, it seems that my initial model with three conv-conv-maxpool sections was the optimal CNN depth for my current configuration.

Model2 Loss Over Epochs

